

Financial Document Intelligence

AI-Powered Financial Document Extraction and Validation

Project Context

Convert financial PDFs and images into structured, validated data with automated financial checks — eliminating manual entry errors and accelerating downstream analysis.

Presented By

Sivaprathish Sivamoorthy

25MDT1150

M.sc data science

VIT Chennai

Problem and Objective

The Challenge

Layout Variability

Financial documents use different layouts and table structures across issuers, making uniform extraction difficult.

Manual Data Entry

Manual transcription takes time and introduces errors that propagate through downstream calculations and reporting.

OCR Dependency

Scanned documents require optical character recognition before any structured information can be processed or extracted.

Validation Gap

Extracted values require numerical checks before further use — a step that is often skipped or performed inconsistently.

The Objective

Build an application that extracts financial information, organizes it into consistent JSON, validates calculations, and stores results for retrieval.

Document Scope

Invoice

Balance Sheet

Profit&Loss

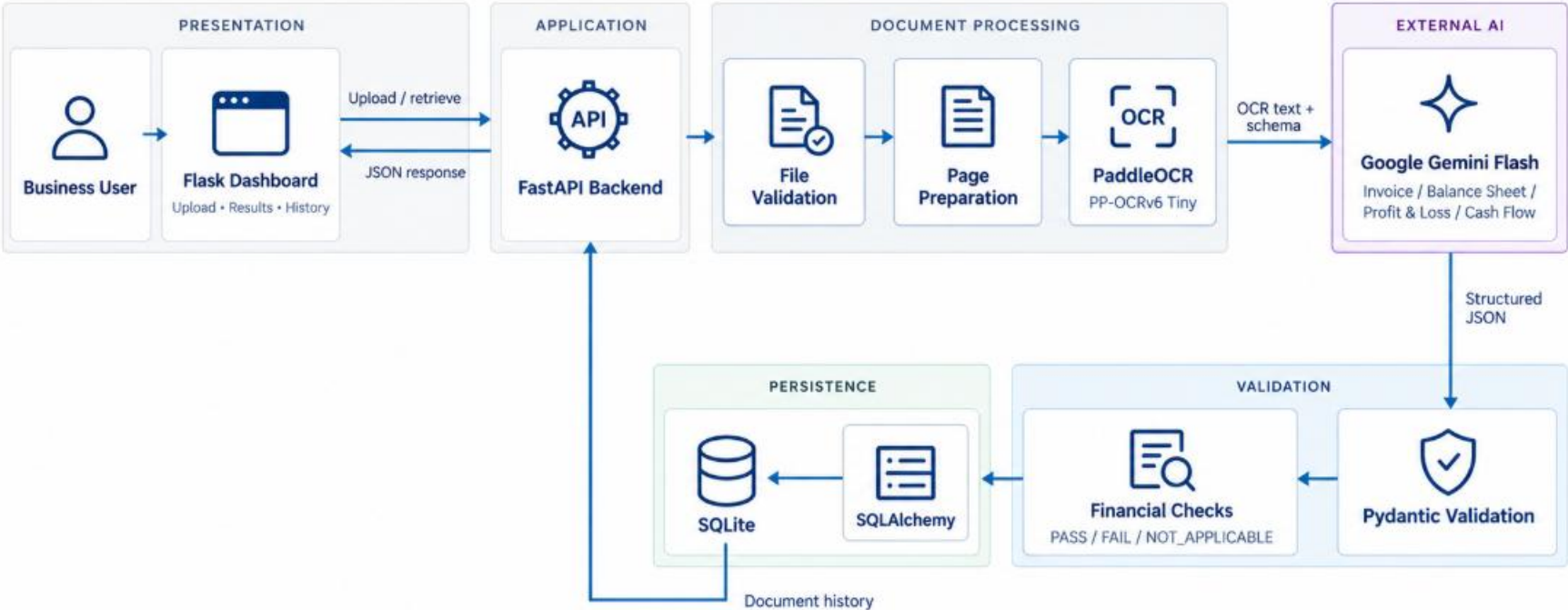
Cash Flow

Solution Architecture

Six integrated layers from user interface to data persistence, each with a clearly defined responsibility.

1	User Interface Flask dashboard for document upload, processing history, and results inspection.
2	Application Layer FastAPI manages incoming requests and coordinates the full document processing workflow.
3	Document Processing File validation, page preparation, and PaddleOCR text extraction from PDFs and images.
4	LLM Extraction Gemini Flash extracts structured information using document-specific schemas from OCR output.
5	Validation Layer Pydantic validates response structure. Deterministic Python rules check all financial calculations.
6	Persistence Layer SQLAlchemy stores and retrieves processing results, metadata, and validation outputs in SQLite.

System Architecture Diagram



Dashboard Overview

FD Document Intelligence

Upload Document history

DOCUMENT WORKSPACE

Turn documents into verified financial data.

Upload a document to extract its fields, review line items, and check the calculations.

Process a document

Document type

Invoice

Choose or drop a document

PDF, JPG, JPEG or PNG · Up to 3 pages · Maximum 10 MB

Choose File

No file chosen

Extract and validate

What you'll receive

- Extracted information**
Fields, line items, and comparative periods.
- Financial checks**
Calculated totals, reported values, and variances.
- Traceable evidence**
Source snippets, page numbers, and downloadable JSON.

Processing can take several minutes. Keep this page open; do not submit the same file again while it is processing.

The Flask dashboard provides a unified interface for document upload, processing history, structured extraction review, and financial validation results — all accessible without direct API interaction.

OCR and LLM Extraction

PaddleOCR — Reading the Document

- Recognizes text and numbers from document images and scanned PDFs.
- Provides full page content as raw text for downstream structured extraction.
- Supplies the exact source material that the LLM uses as its reference input.

Gemini Flash — Interpreting the Content

- Identifies document fields and financial line items from unstructured OCR text.
- Associates values with their correct reporting periods across multi-period documents.
- Returns structured JSON conforming to the selected document-specific schema.

Key principle: The LLM extracts information. Deterministic code checks calculations. These responsibilities are intentionally separated to ensure auditability and reproducibility.

Technology Stack

Layer	Technology	Purpose
Frontend	Flask, HTML, CSS, JavaScript	Document upload, processing history, and results dashboard
Backend	FastAPI	REST APIs and document-processing orchestration
OCR	PaddleOCR	Extract text and numbers from document images
LLM	Google Gemini Flash	Convert OCR text into structured financial data
Schema Validation	Pydantic	Validate document-specific schemas and response structure
Financial Validation	Python	Check financial calculations using configured numerical tolerances
Database	SQLite	Store document metadata, extracted data, and validation results
Database Access	SQLAlchemy	Manage database operations and result retrieval
API Documentation	Swagger UI / OpenAPI	Document and interactively test backend endpoints
Development Tools	uv, Uvicorn	Manage dependencies and run the FastAPI application

APIs, Dashboard, and Persistence

REST API Endpoints

Endpoint	Function
POST /api/v1/documents/process	Upload and process a document
GET /api/v1/documents/{name}	Retrieve a document by name
GET /api/v1/documents	List all processed documents
/docs	Interactive Swagger documentation

Dashboard & Persistence

Flask Dashboard

Upload documents, browse processing history, inspect extracted values, and review financial validation checks – all through a single unified interface.

SQLAlchemy + SQLite

SQLAlchemy manages all database access. SQLite stores document metadata, structured extraction output, and validation results for later retrieval and audit.

Document-Specific Schemas & Common Output Design

Extraction Targets by Document Type

Document	Extraction Targets
Invoice	Invoice number, date, vendor, customer, line items, tax, and total
Balance Sheet	Assets, liabilities, equity, totals, and reporting periods
Profit and Loss	Income, expenses, profit figures, and comparative periods
Cash Flow Statement	Operating, investing, financing, and cash reconciliation values

Common Output Design Principles

Dual Value Representation Retain the printed value alongside its normalized numeric value for traceability.	Period Preservation Preserve separate values for each reporting period – no collapsing of comparative columns.
Null for Missing Data Represent missing or unextracted information with <code>null</code> rather than a default or zero.	Extraction Warnings Include extraction warnings where ambiguity or low-confidence readings are detected.

Financial Validation

Illustrative Invoice Check

Field	Value
Subtotal	12,500.00
Tax	625.00
Discount	0.00
Reported Total	13,125.00
Calculated Total	13,125.00
Absolute Variance	0.00 ✓

Validation Formula

$$\text{Total} = \text{Subtotal} + \text{Tax} - \text{Discount}$$

Example tolerance: **±0.01** in the same units as the compared values.

Validation Outcome States

PASS

Difference falls within the configured tolerance threshold.

FAIL

Difference exceeds the configured tolerance — flagged for review.

NOT_APPLICABLE

Required inputs are unavailable or the rule does not apply to this document.

Each validation result records the **formula, inputs, calculated value, reported value, variance, and status** — providing a full audit trail for every check performed.

Thank You
